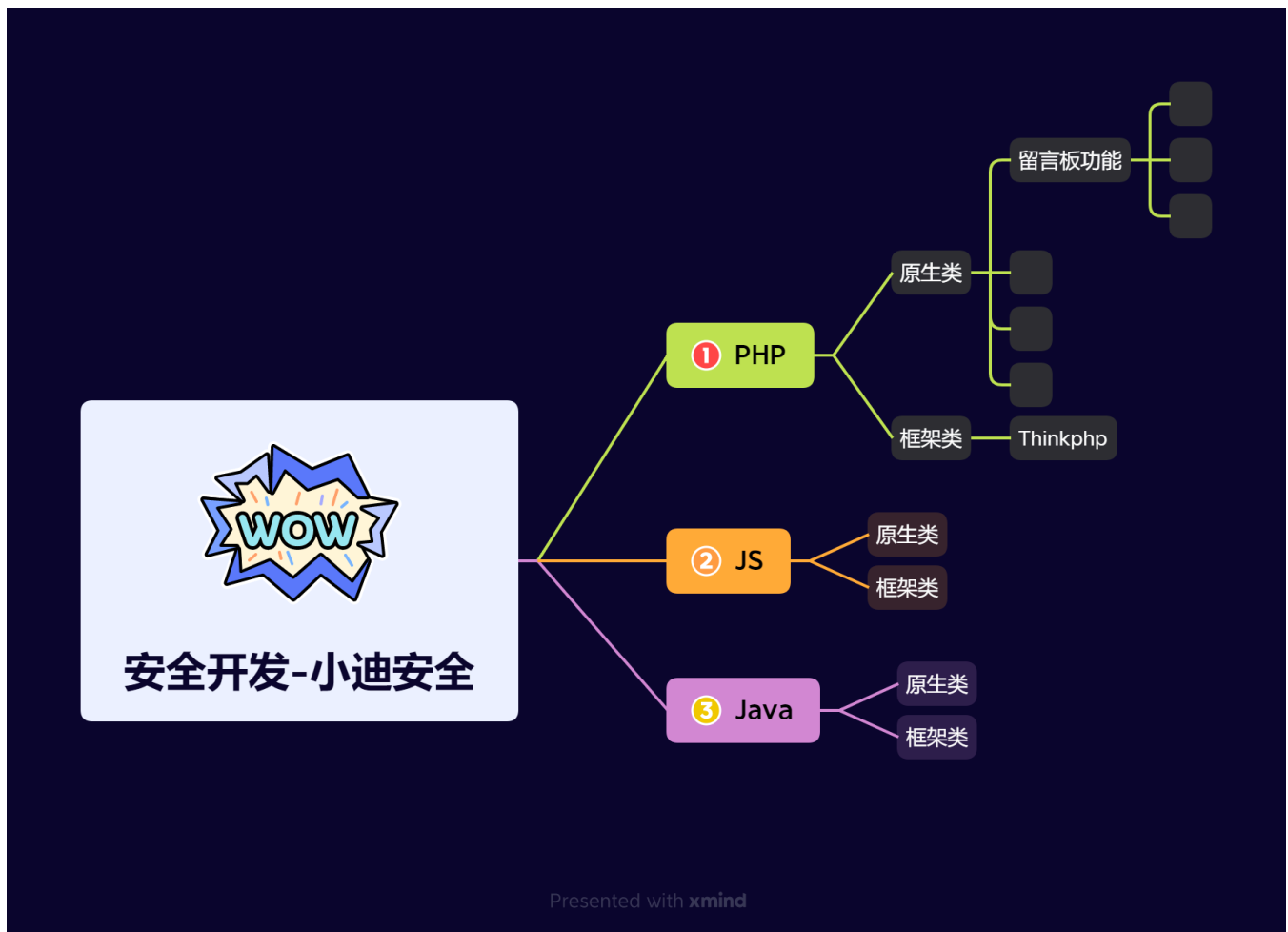


# Day23 安全开发 - PHP 应用&后台模块 &Session&Cookie&Token&身份验证&唯一性

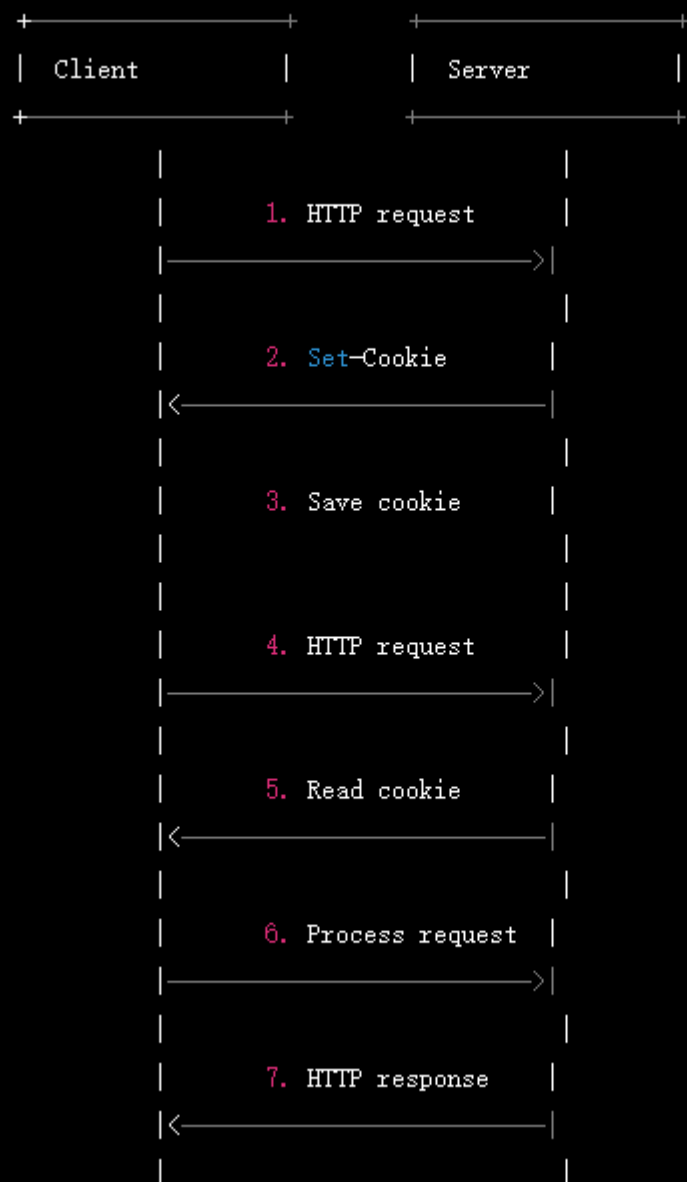


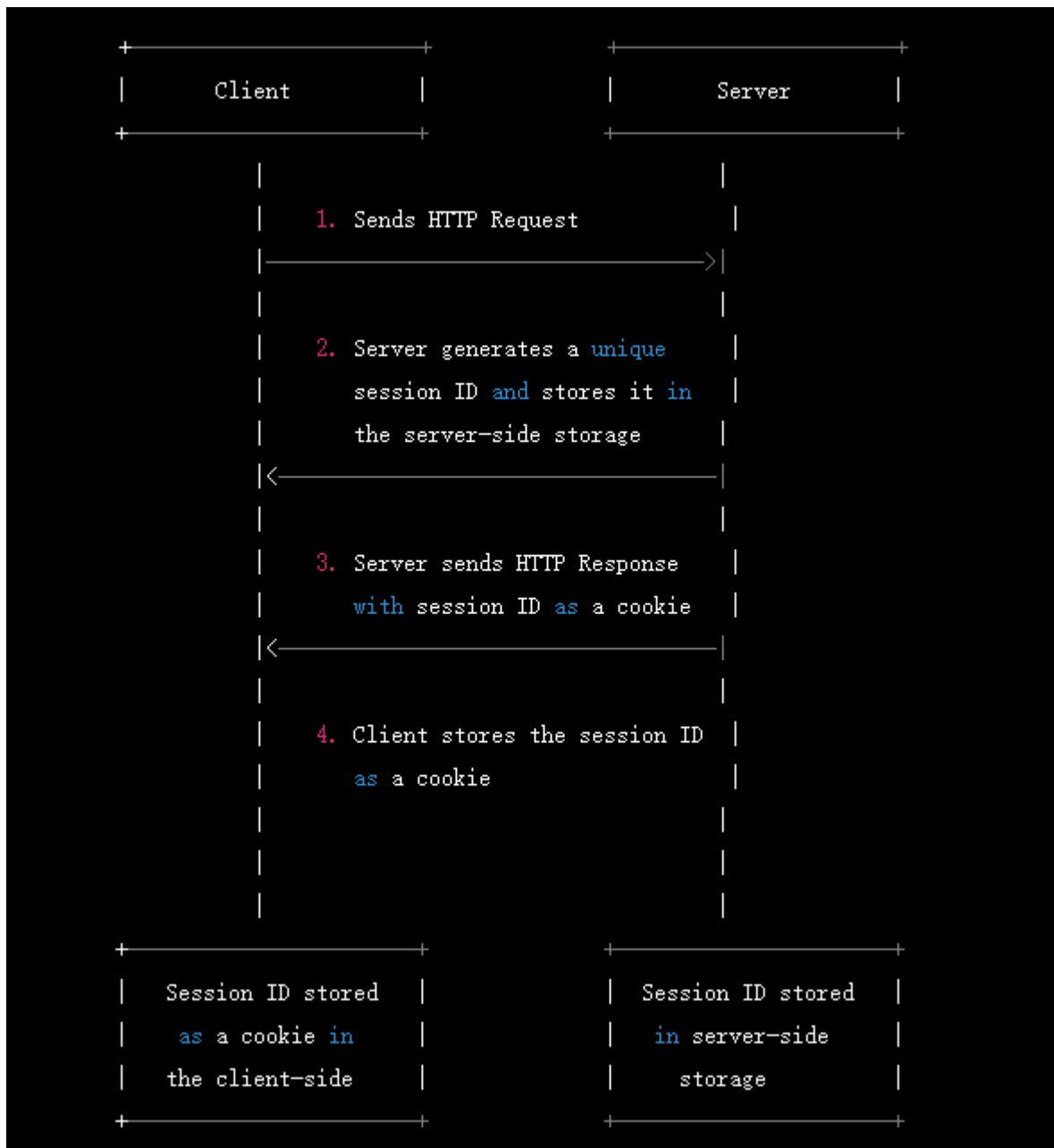
## 1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

- 
- 1、PHP后台身份验证模块实现
  - 2、Cookie&Session技术&差异
  - 3、Token数据包唯一性应用场景
  - 项目1：用cookie做后台身份验证
  - 项目2：用session做后台身份验证
  - 项目3：用token做用户登录判断

## 2.演示案例





- 1 #具体安全知识点:
- 2 -Cookie和Session都是用来在web应用程序中跟踪用户状态的机制
- 3 1、存储位置不同:
- 4 Cookie是存储在客户端（浏览器）上的，而Session是存储在服务器端的。
- 5 2、安全性不同:
- 6 Cookie存储在客户端上，可能会被黑客利用窃取信息，而Session存储在服务器上，更加安全。
- 7 3、存储容量不同:
- 8 Cookie的存储容量有限，一般为4KB，而Session的存储容量理论上没有限制，取决于服务器的硬件和配置。
- 9 4、生命周期不同:
- 10 Cookie可以设置过期时间，即便关闭浏览器或者重新打开电脑，Cookie仍然存在，直到过期或者被删除。而Session一般默认在浏览器关闭后就会过期。
- 11 5、访问方式不同:
- 12 Cookie可以通过JavaScript访问，而Session只能在服务器端进行访问。
- 13 6、使用场景不同:

14 **Cookie**一般用于存储小型的数据，如用户的用户名和密码等信息。而**Session**一般用于存储大型的数据，如购物车、  
登录状态等信息。

15 总之，**Cookie**和**Session**都有各自的优缺点，选择使用哪一种方式，取决于具体的应用场景和需求。一般来说，如果  
需要存储敏感信息或者数据较大，建议使用**Session**；如果只需要存储少量的数据，并且需要在客户端进行访问，可  
以选择使用**Cookie**。

16

17 -在web应用程序中，使用**token**和不使用**token**的主要差异在于身份验证和安全性。

18 1. 身份验证：采用**token**机制的web应用程序，用户在登录成功后会收到一个**token**，这个**token**可以在每次请求时  
发送给服务器进行身份验证。而不采用**token**机制的web应用程序，一般会使用**session**机制来保存用户登录状态，  
服务器会在用户登录成功后创建一个**session**，之后的每个请求都需要在HTTP头中附带这个**session ID**，以便服务  
器能够验证用户身份。

19 2、安全性：采用**token**机制的web应用程序，在服务器上不会存储用户的登录状态，只需要存储**token**即可。因此，  
即使**token**被盗取，黑客也无法获得用户的密码或者其他敏感信息。而不采用**token**机制的web应用程序，一般会在服  
务器上存储用户的登录状态，因此如果服务器被黑客攻击，黑客可能会获得用户的敏感信息。

20 3、跨域访问：采用**token**机制的web应用程序，在跨域访问时，可以使用HTTP头中的**Authorization**字段来传递  
**token**信息，方便实现跨域访问。而不采用**token**机制的web应用程序，在跨域访问时，需使用**cookie**或**session**来  
传递用户身份信息，比较麻烦。

21 总之，采用**token**机制可以提高web应用程序的安全性，并且方便实现跨域访问。不过，使用**token**机制也需要开发者  
自己来实现身份验证和**token**的生成和验证，相对来说比较复杂。而不采用**token**机制，使用**session**机制则相对简  
单，但是安全性相对较低。因此，具体采用哪种机制，需要根据实际情况进行权衡和选择。

## 2.1 后台模块-身份验证-Cookie技术

● ● ●

1 生成**cookie**的原理图过程：见图

2 1、客户端向服务器发送HTTP请求。

3 2、服务器检查请求头中是否包含**cookie**信息。

4 3、如果请求头中包含**cookie**信息，则服务器使用该**cookie**来识别客户端，否则服务器将生成一个新的**cookie**。

5 4、服务器在响应头中设置**cookie**信息并将其发送回客户端。

6 5、客户端接收响应并将**cookie**保存在本地。

7 6、当客户端发送下一次HTTP请求时，它会将**cookie**信息附加到请求头中。

8 7、服务器收到请求并检查**cookie**的有效性。

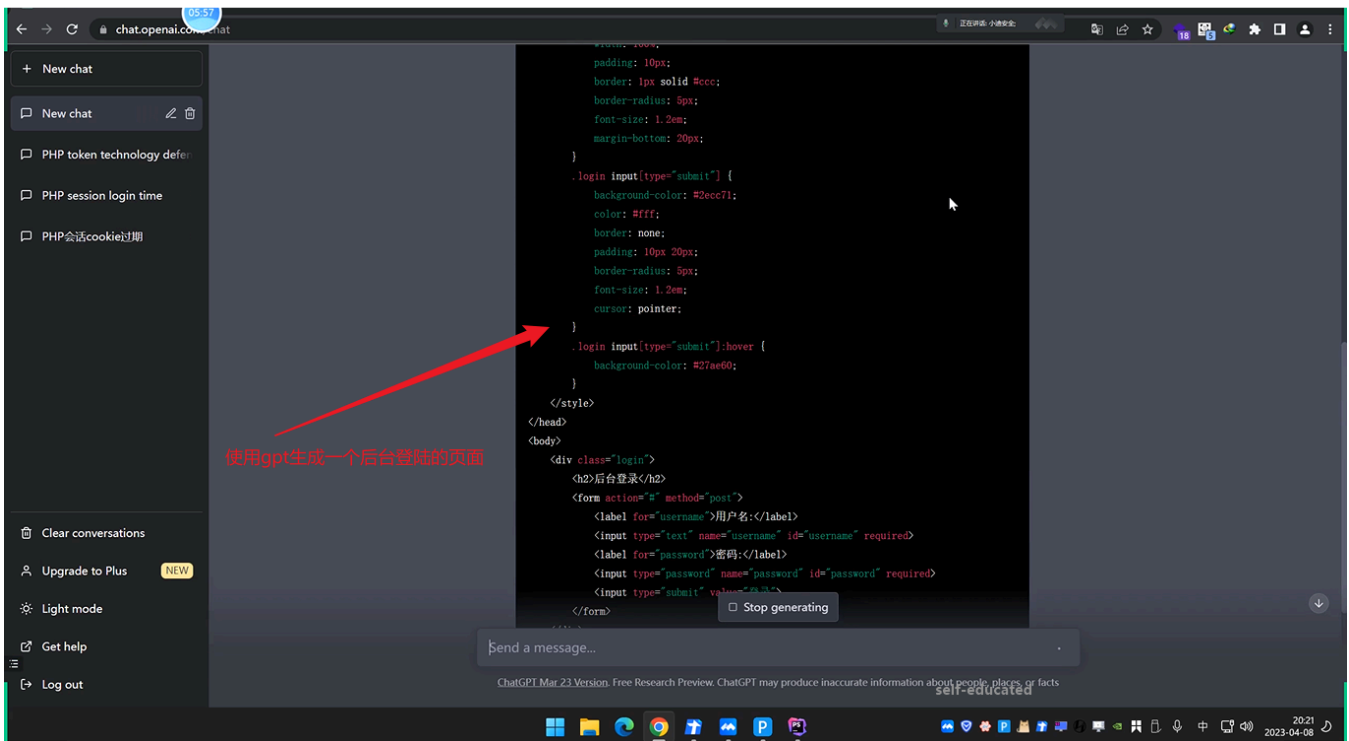
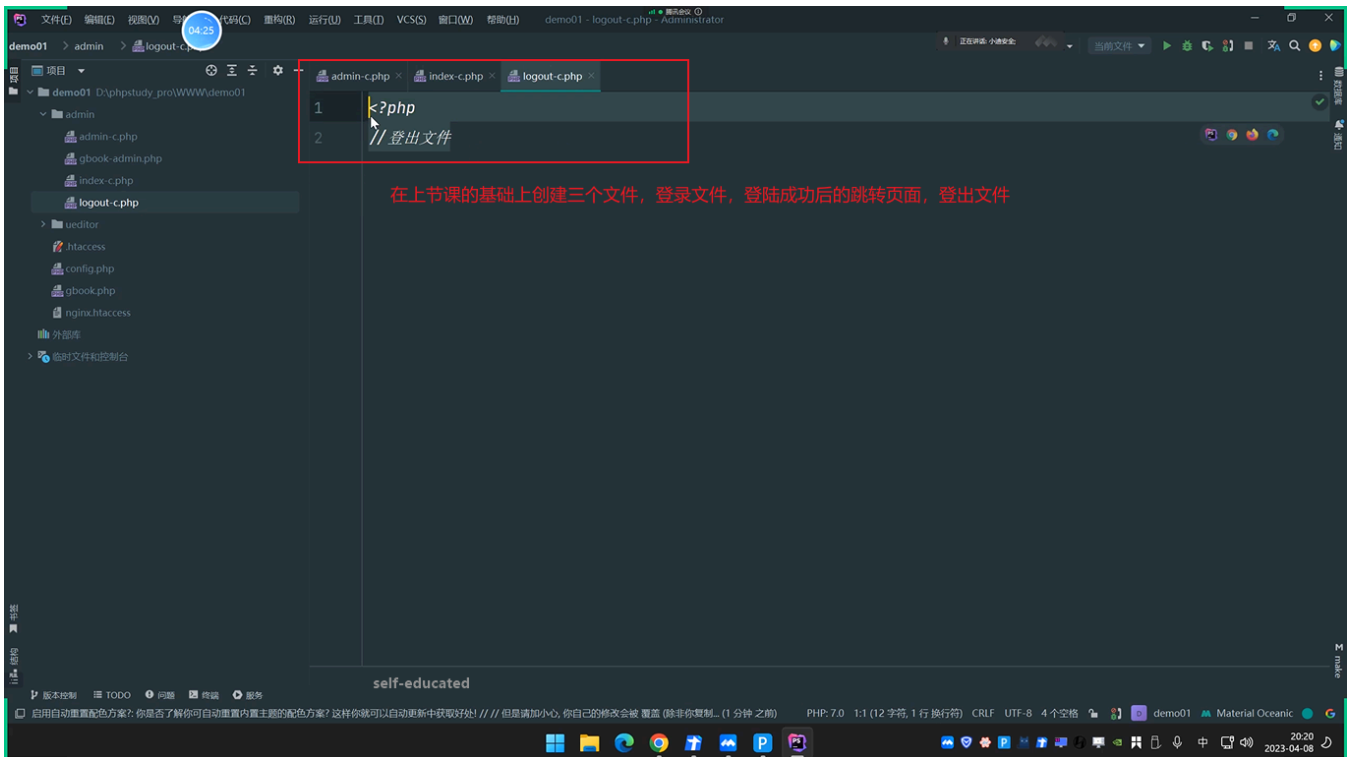
9 8、如果**cookie**有效，则服务器响应请求。否则，服务器可能会要求客户端重新登录。

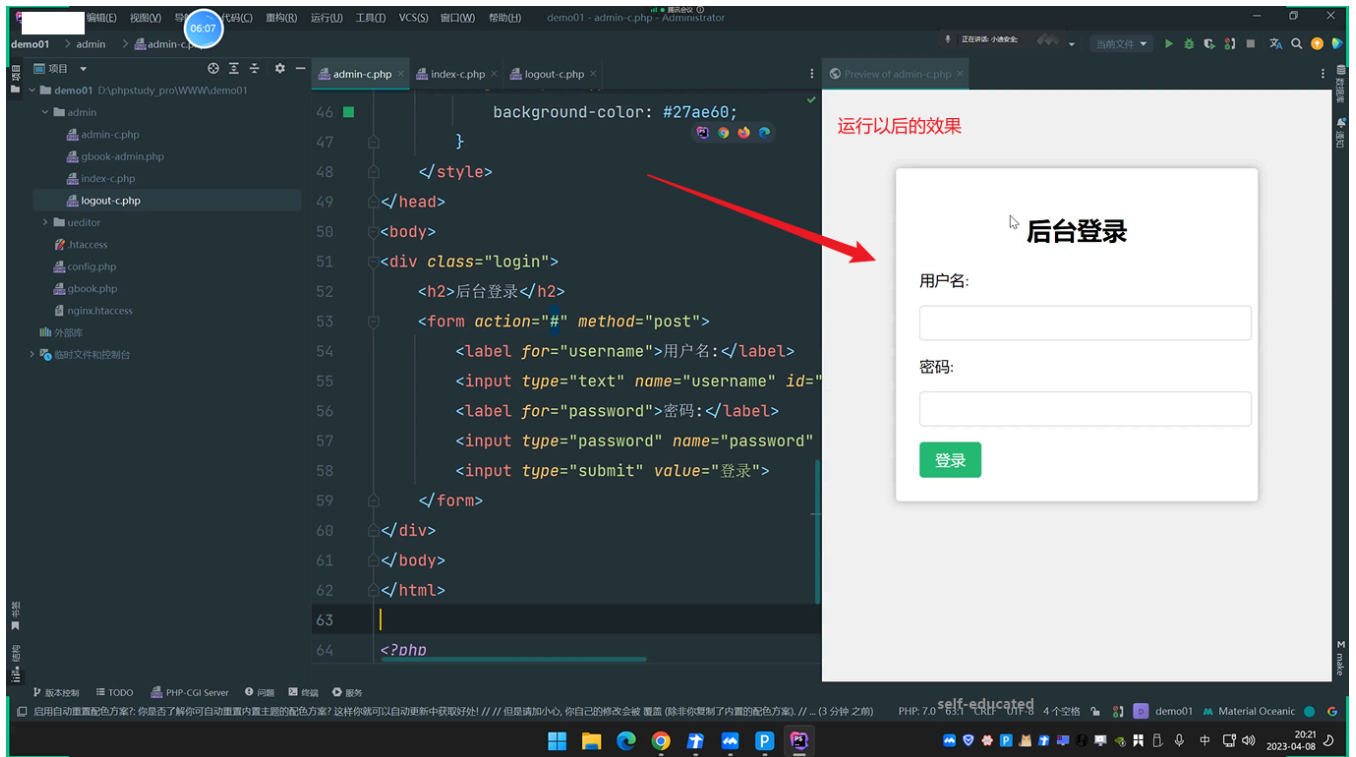
10

11 **setcookie()**：设置一个**cookie**并发送到客户端浏览器。

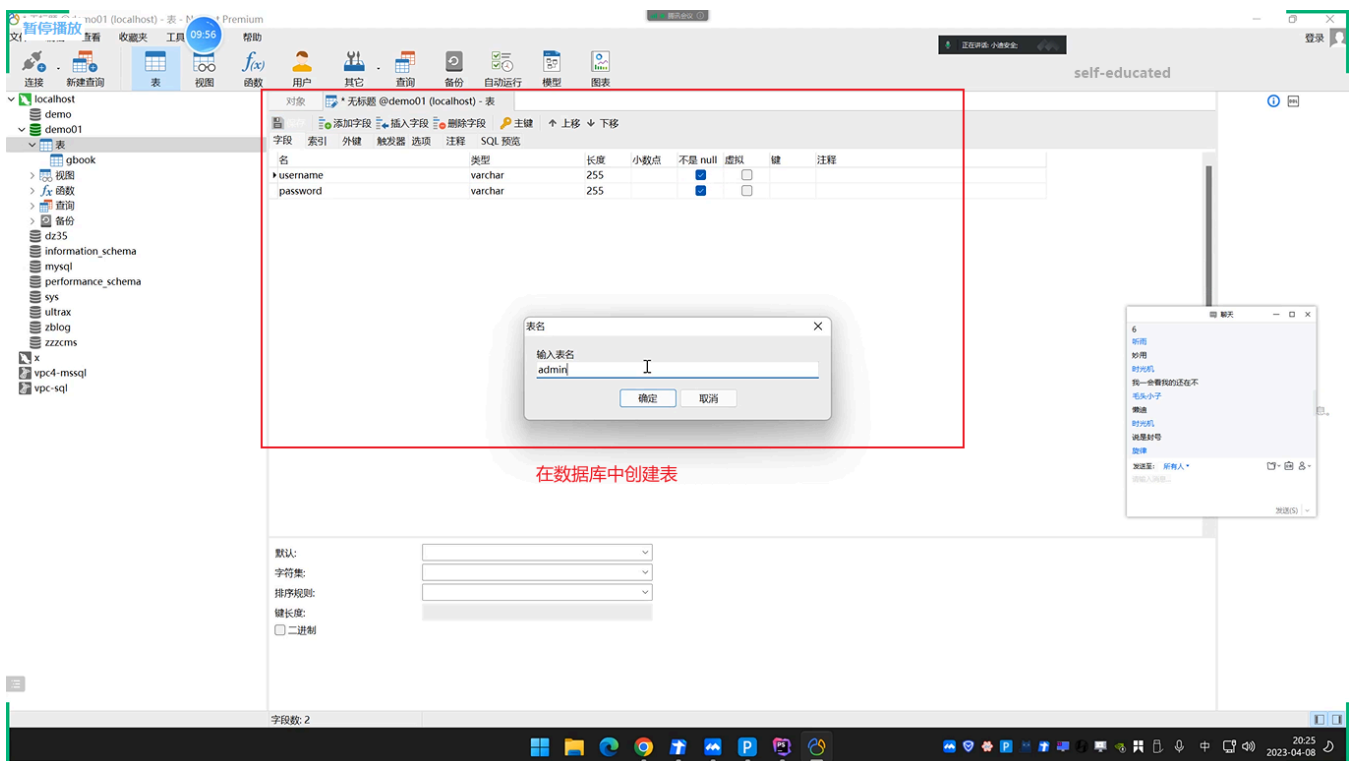
12 **unset()**：用于删除指定的**cookie**。

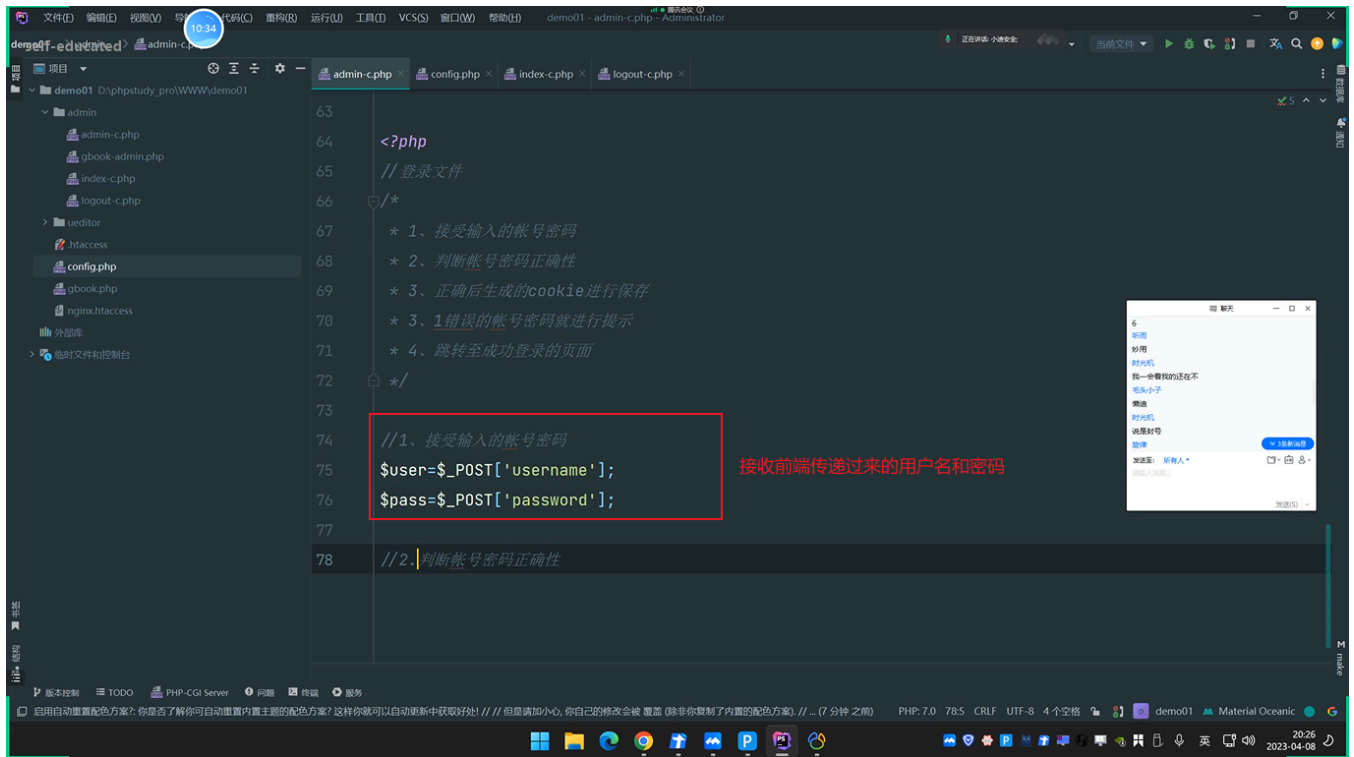
## (1) 后台登录模块:



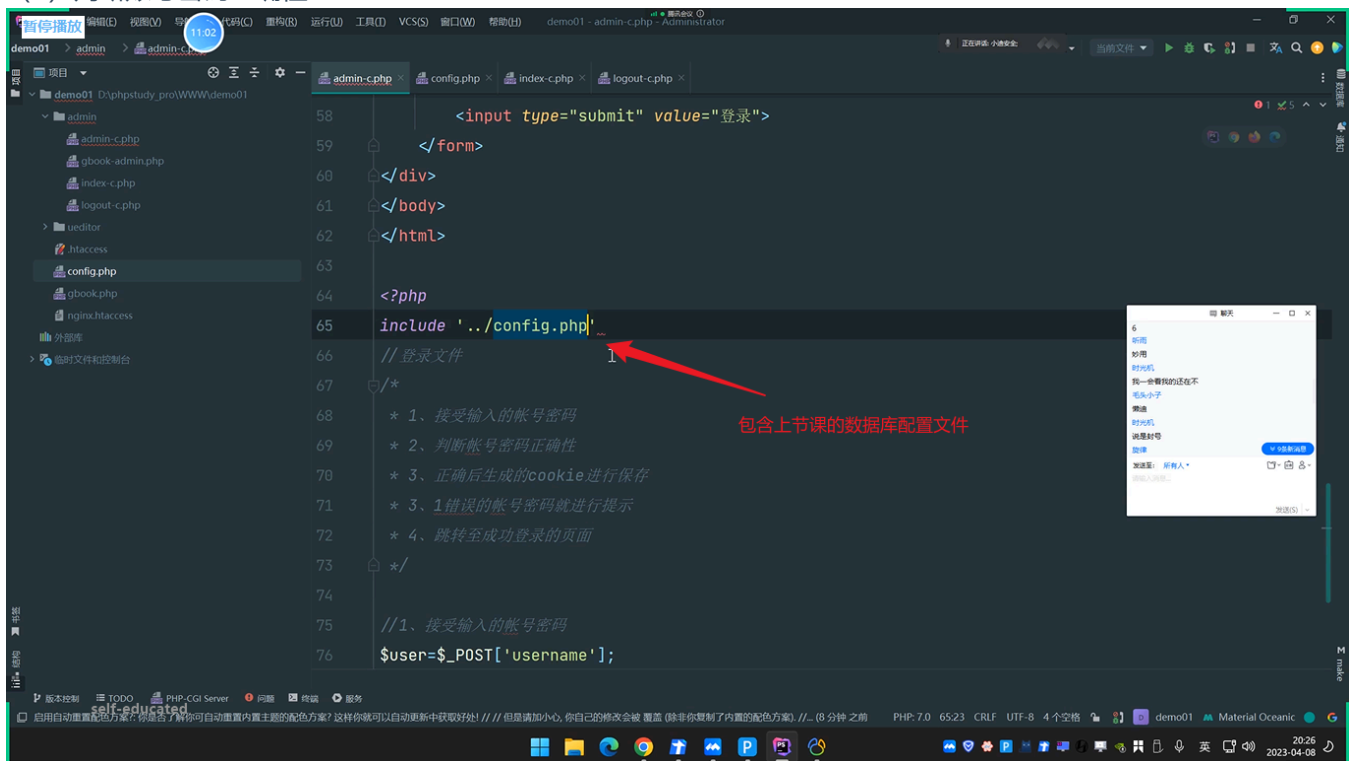


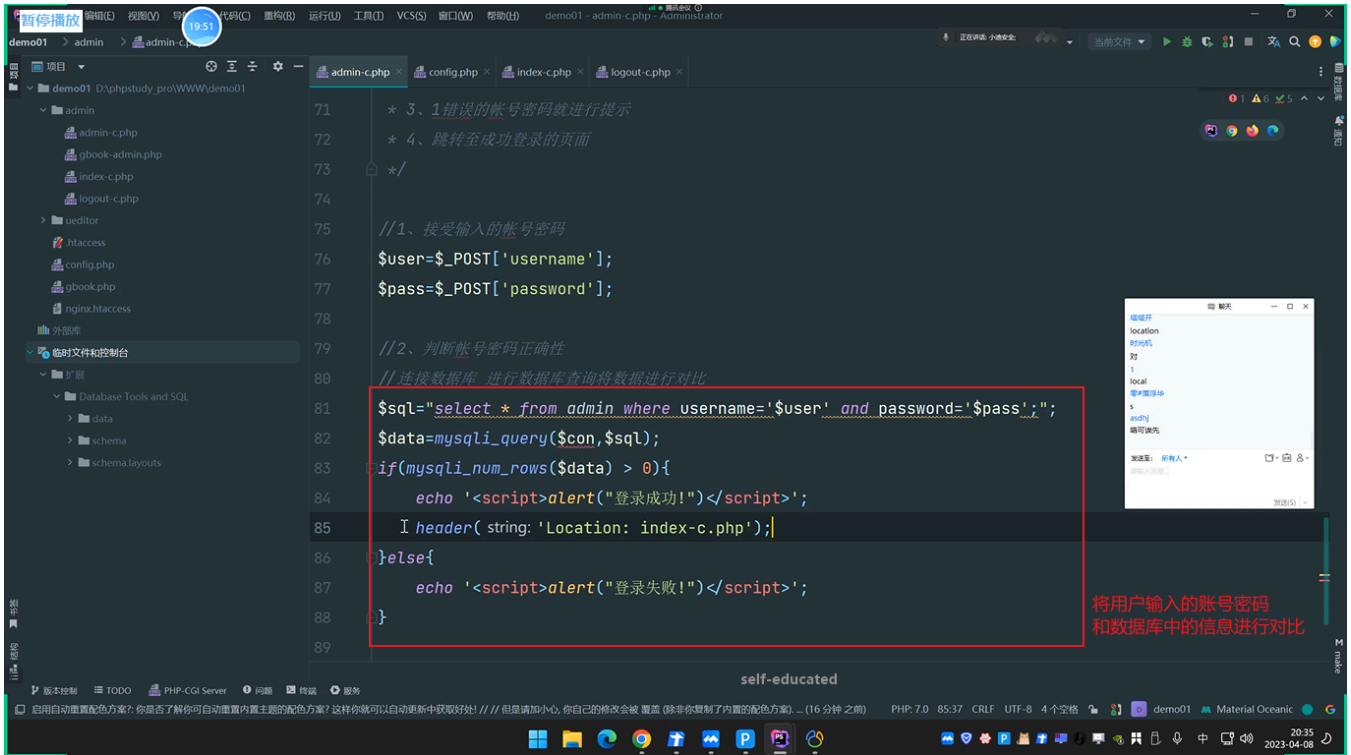
## (2) 接收输入的账号密码:



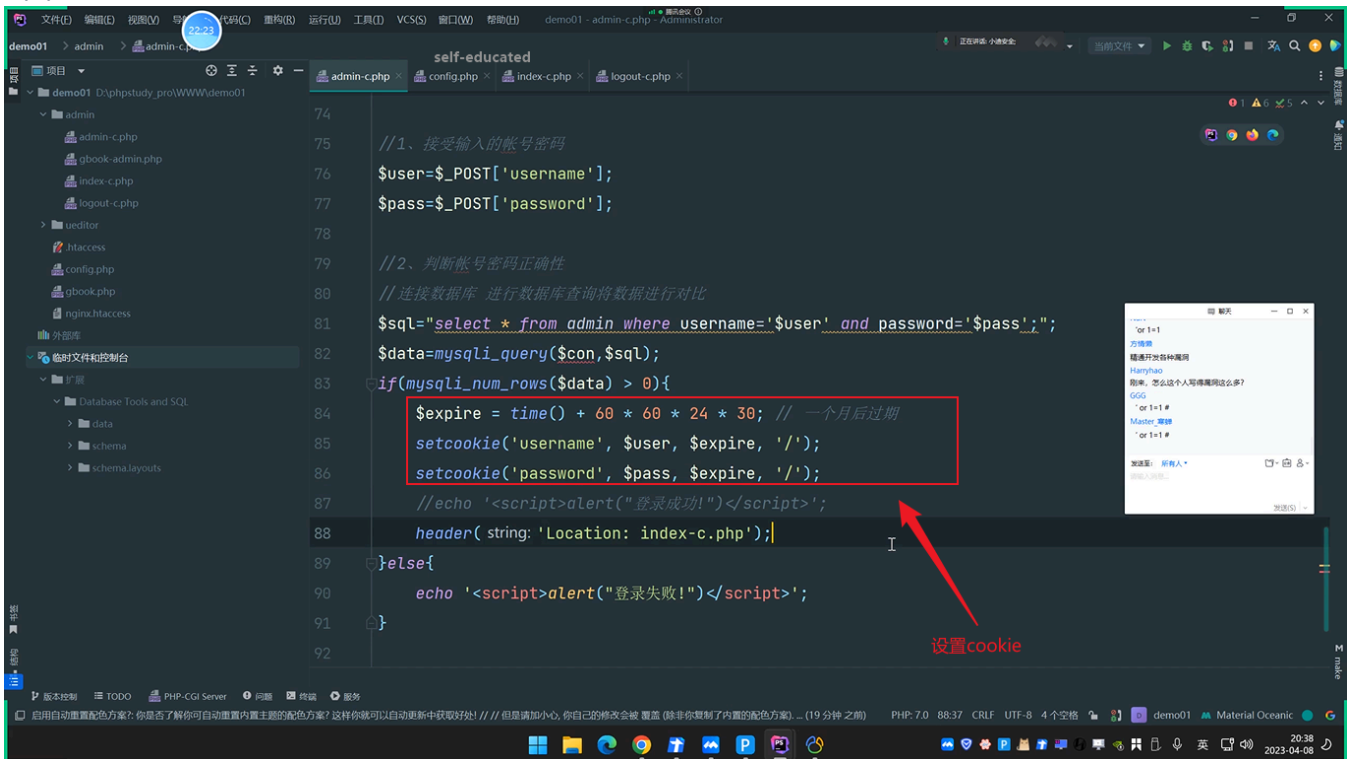


### (3) 判断账号密码正确性:





#### (4) 正确登录后，生成cookie进行保存：



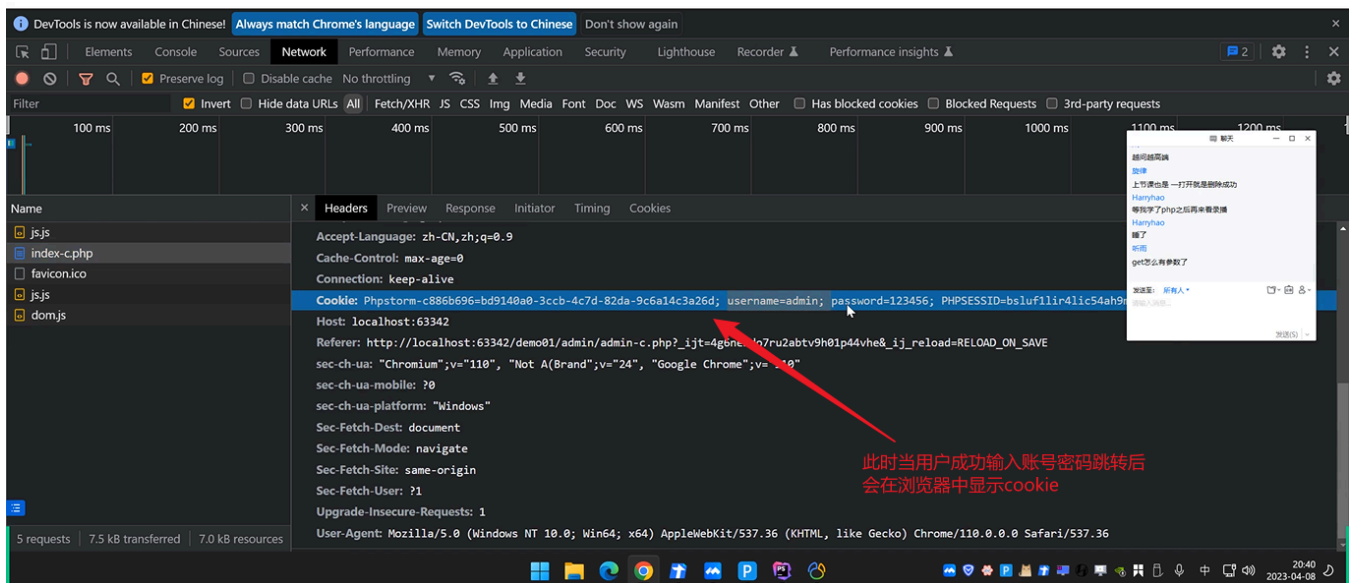
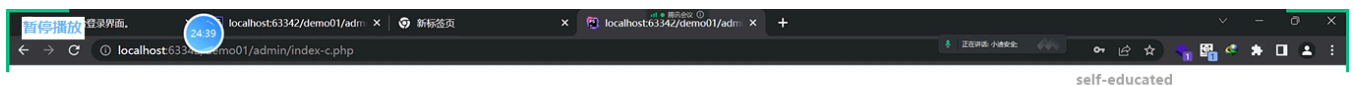
demo01 - admin - Administrator

admin-c.php

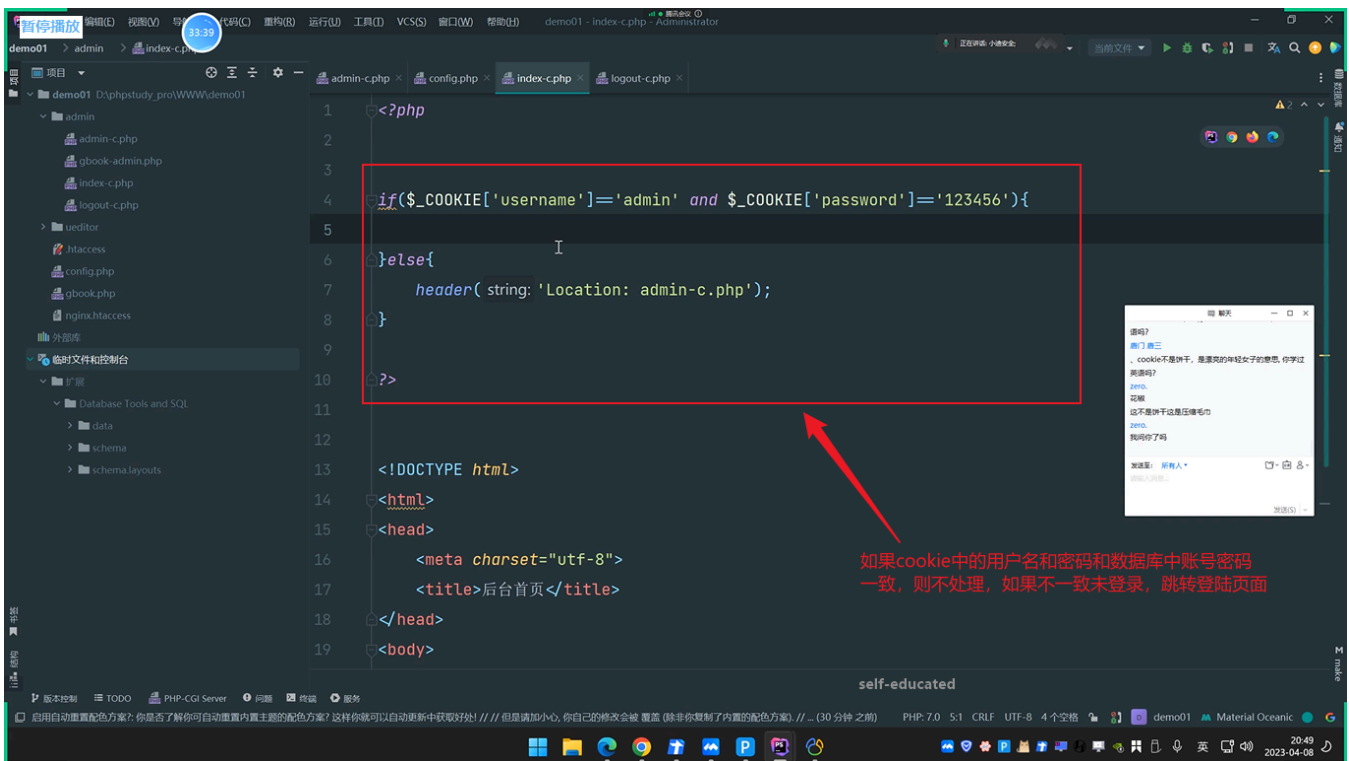
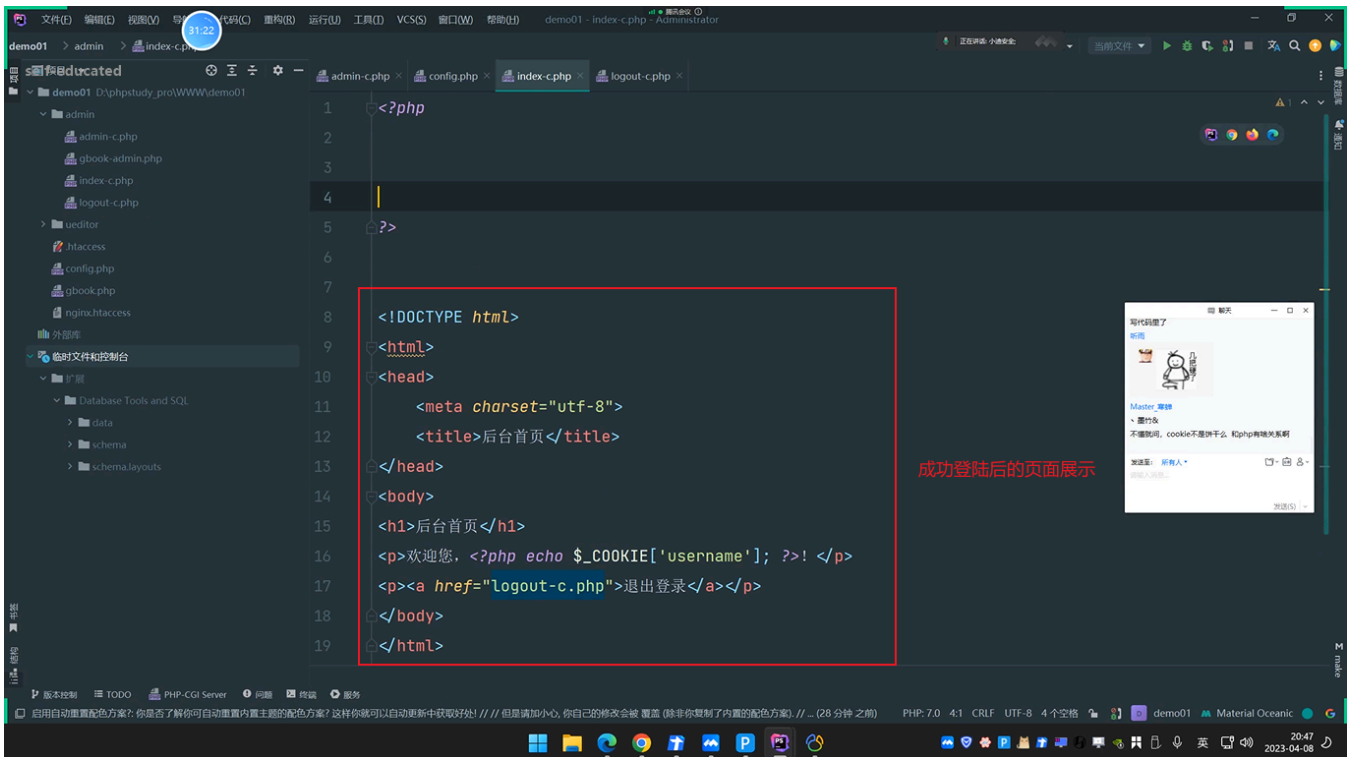
```
76 $user=$_POST['username'];
77 $pass=$_POST['password'];
78
79 //2、判断账号密码正确性
80 //连接数据库 进行数据库查询将数据进行对比
81 $sql="select * from admin where username='$user' and password='$pass'";
82 $data=mysqli_query($con,$sql);
83 if($_SERVER["REQUEST_METHOD"] == "POST"){
84     if(mysqli_num_rows($data) > 0){
85         $expire = time() + 60 * 60 * 24 * 30; // 一个月后过期
86         setcookie('username', $user, $expire, '/');
87         setcookie('password', $pass, $expire, '/');
88         //echo '<script>alert("登录成功!")</script>';
89         header( string: 'Location: index-c.php'); I
90         exit();
91     }else{
92         echo '<script>alert("登录失败!")</script>';
93     }
94 }
```

self-educated

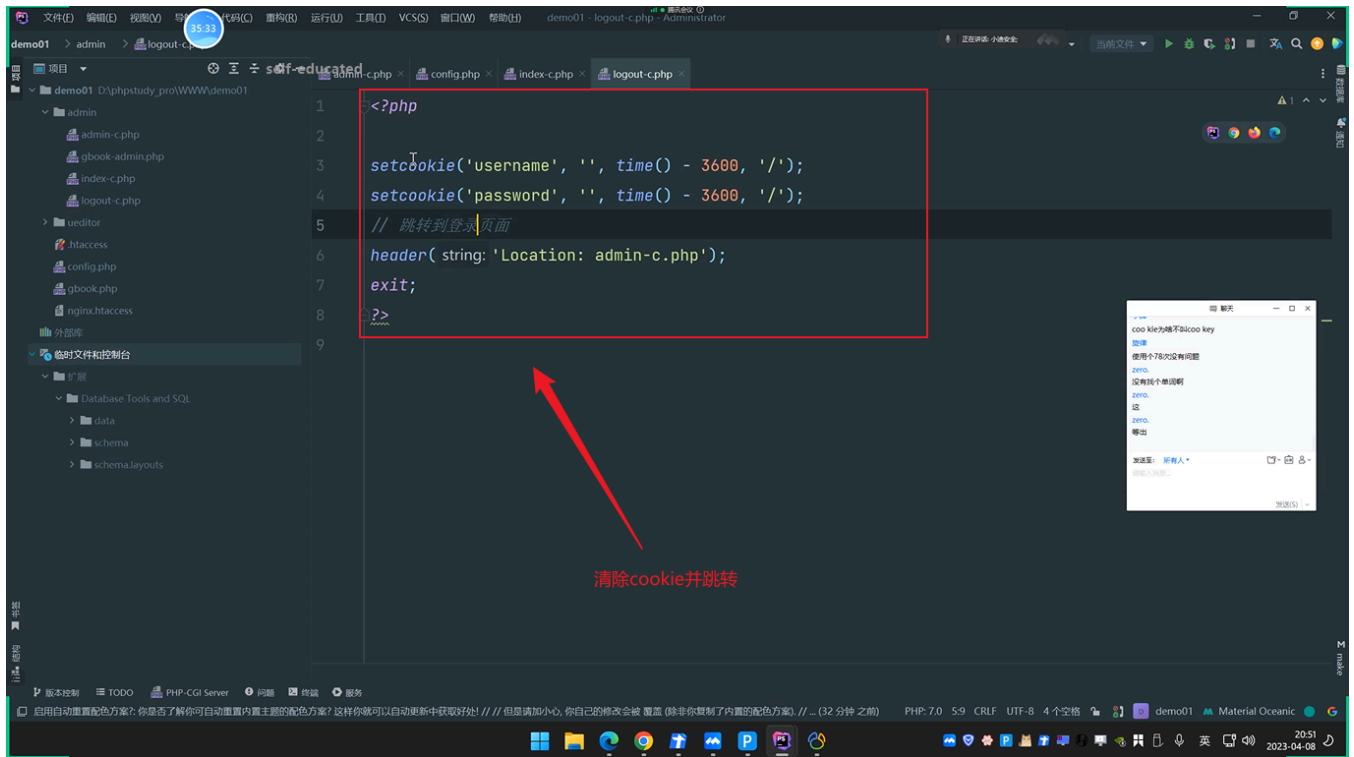
为了避免一直弹窗登陆失败  
可以对表单是否提交进行判断  
当提交后才执行后续代码



##### (5) 对跳转后的页面进行判断:



##### (6) 登出页面设计:

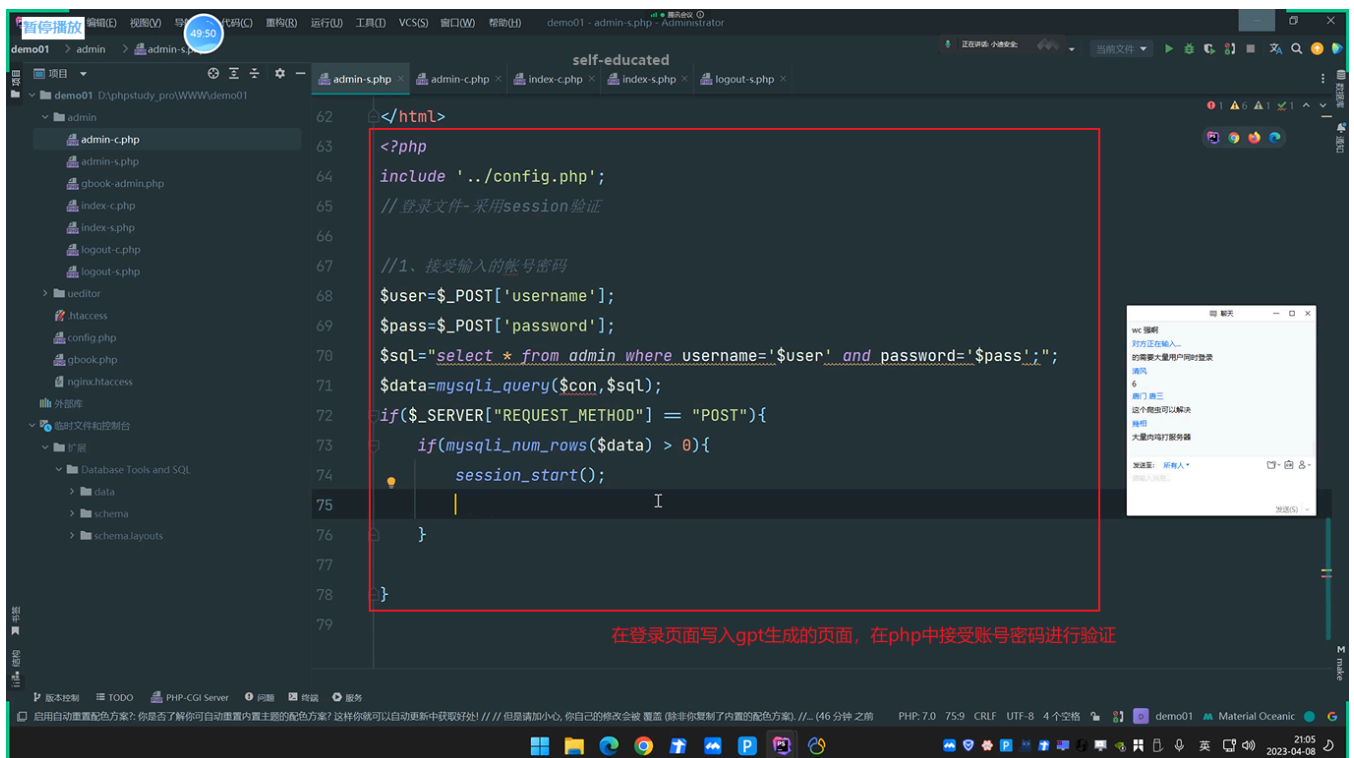
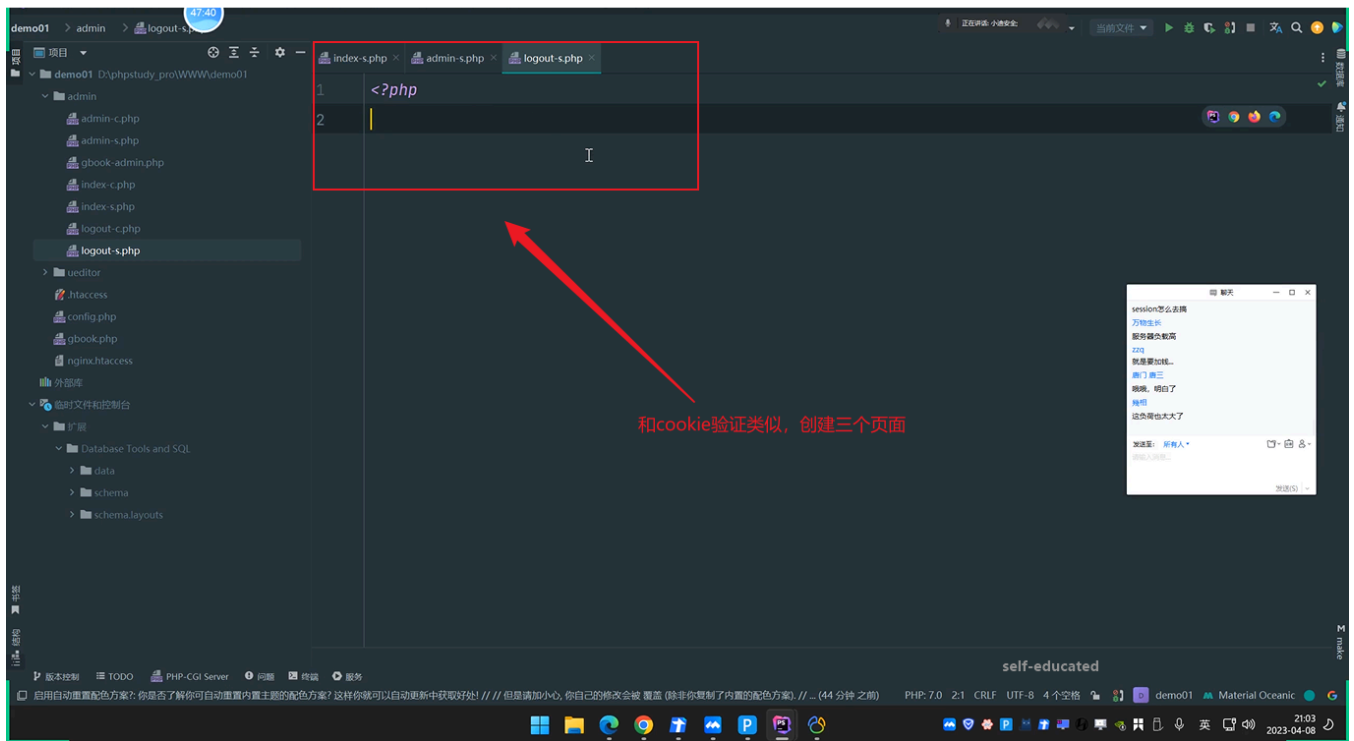


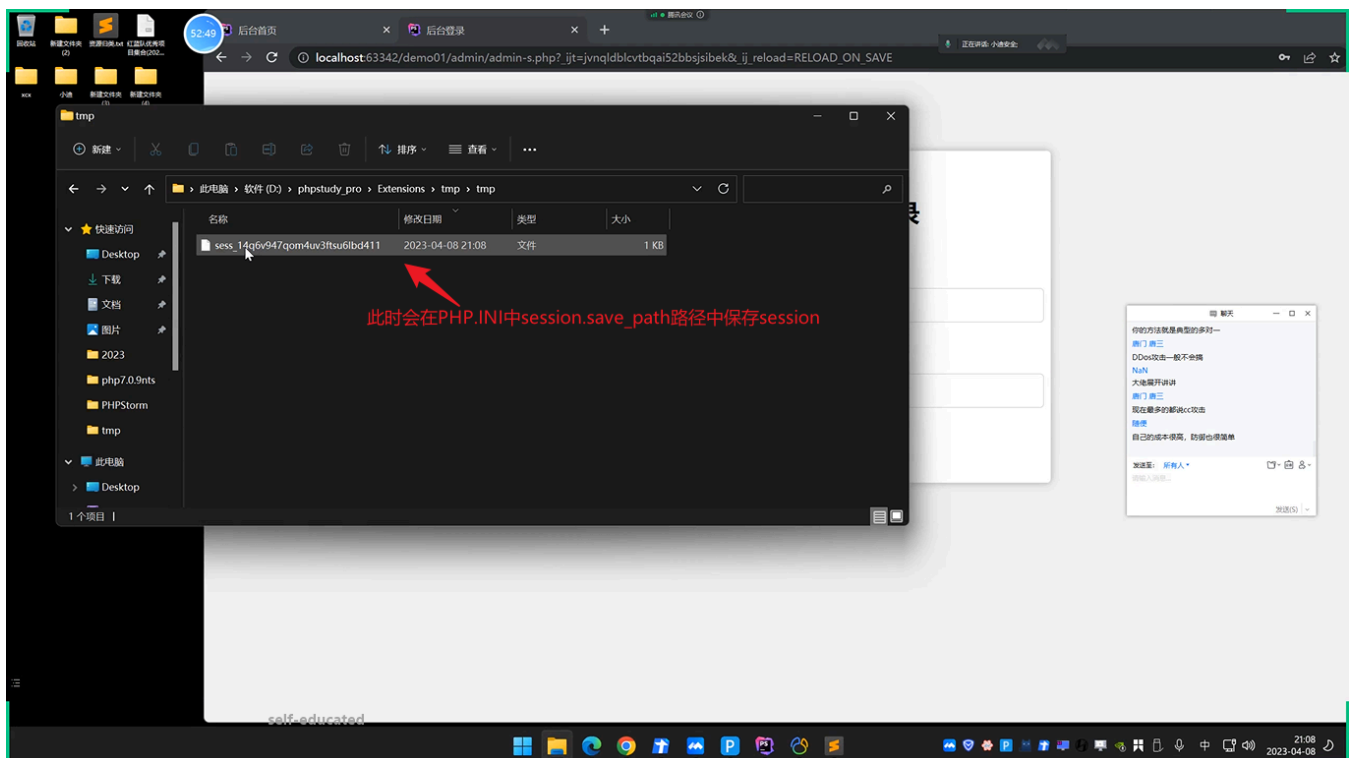
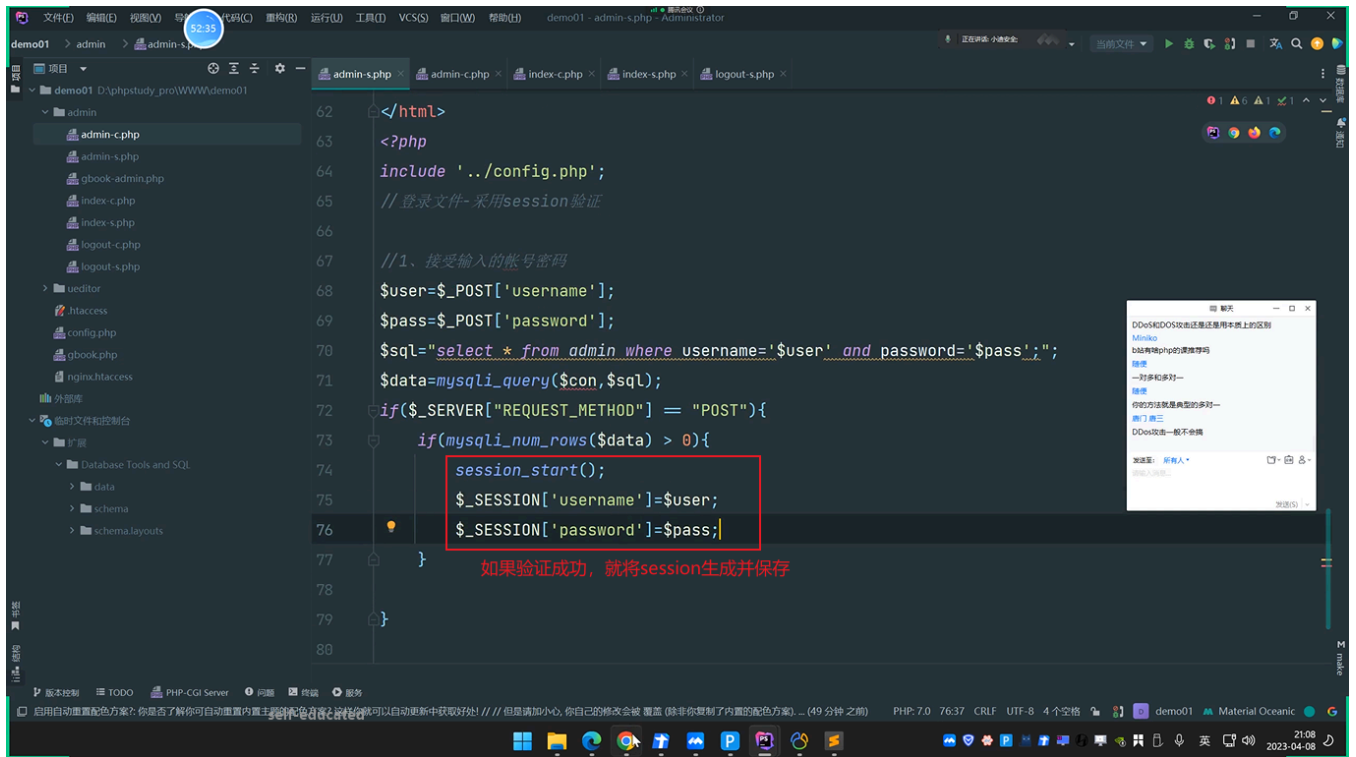
- 1 存在问题：当用户获取到cookie就能假冒用户登录。

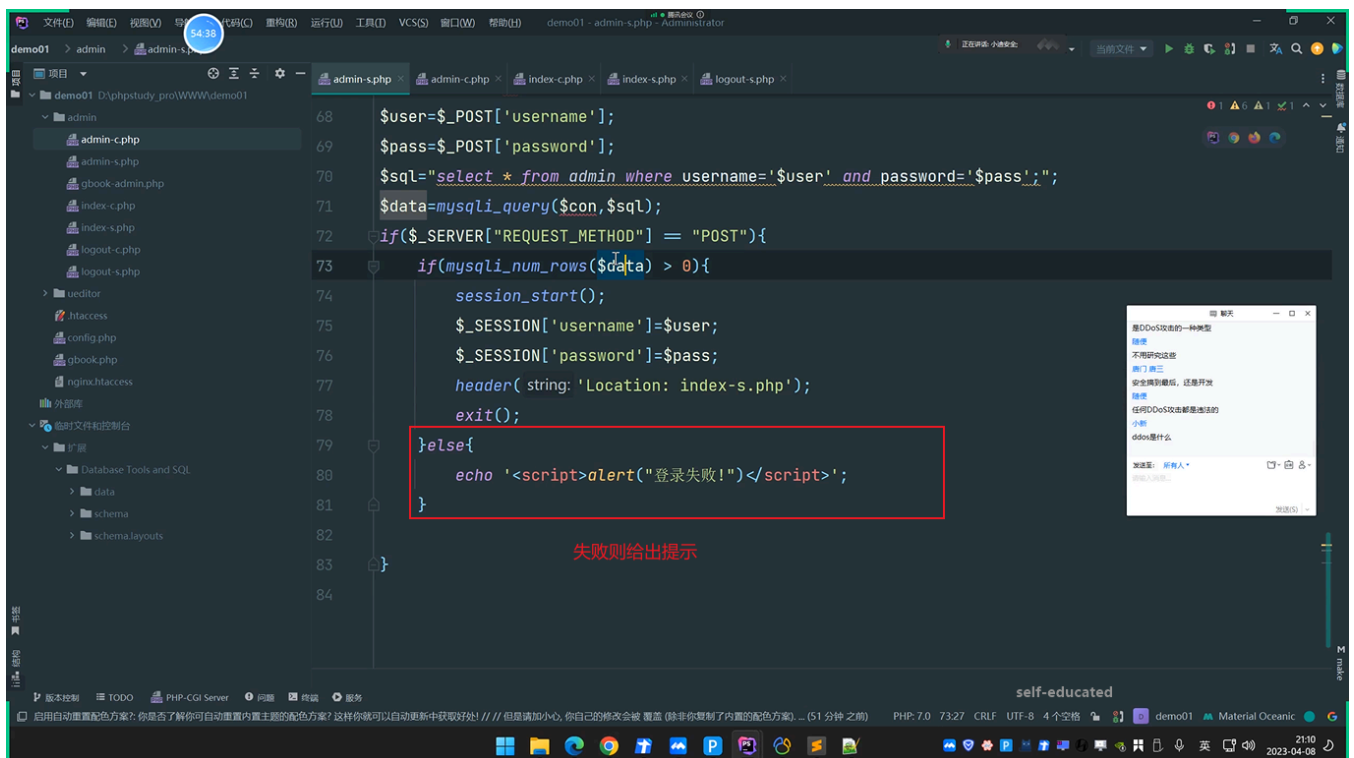
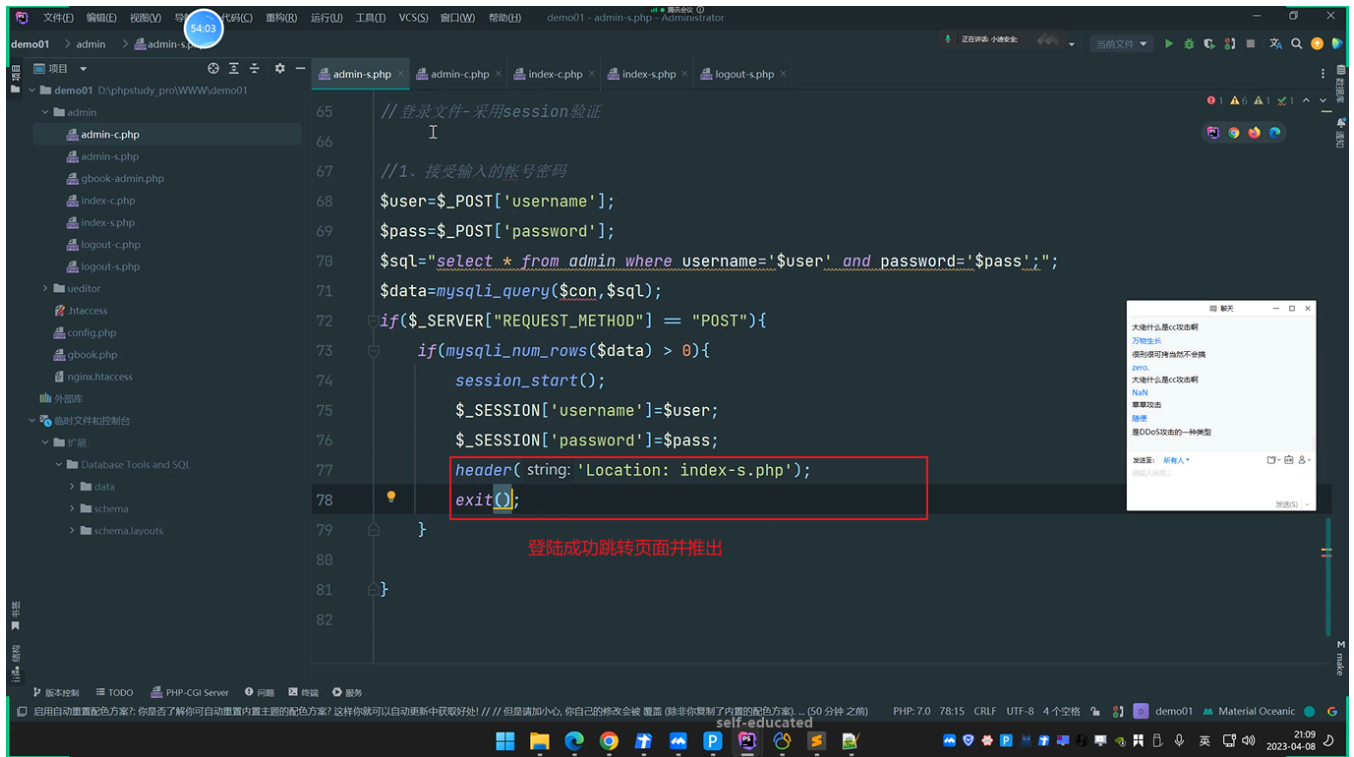
## 2.2 后台模块-身份验证-Session技术

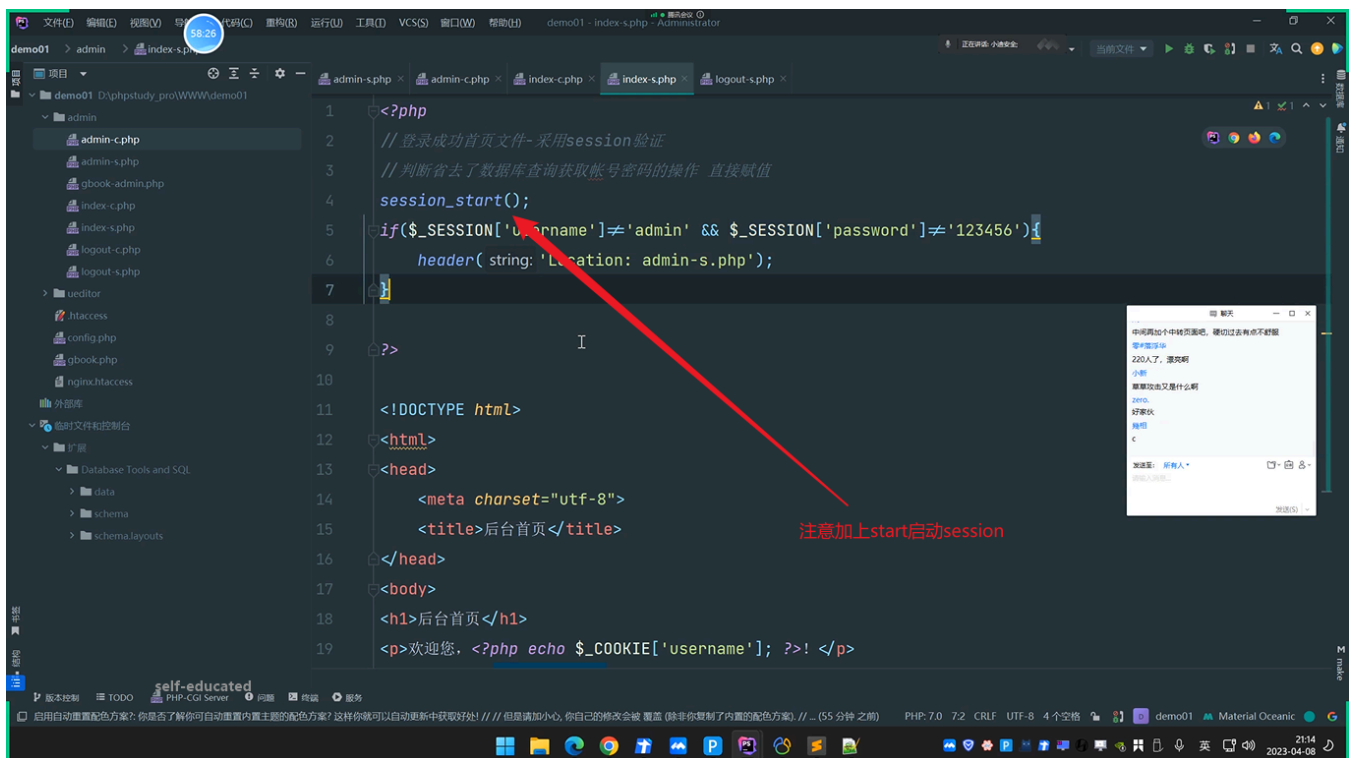
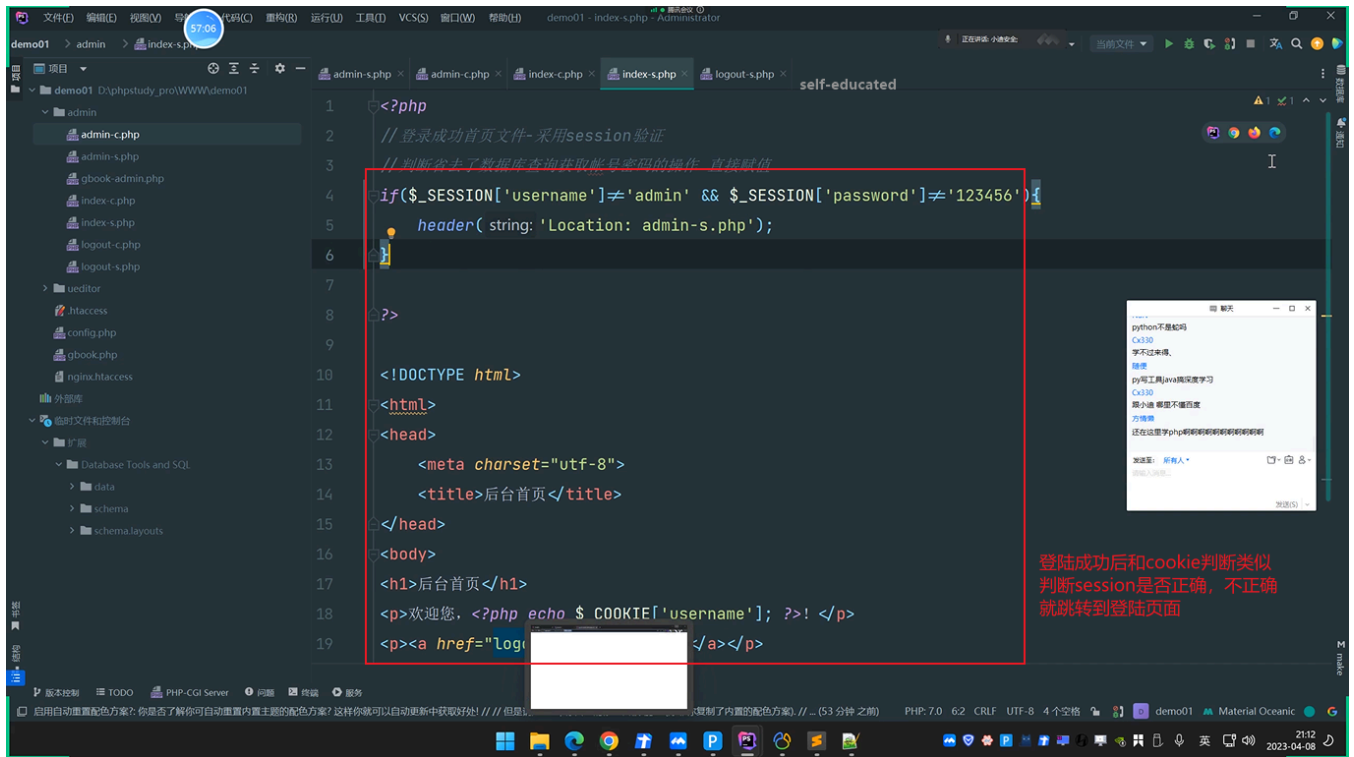
- 1 1、客户端向服务器发送HTTP请求。
- 2 2、服务器为客户端生成一个唯一的session ID，并将其存储在服务器端的存储器中（如文件、数据库等）。
- 3 3、服务器将生成的session ID作为一个cookie发送给客户端。
- 4 4、客户端将session ID保存为一个cookie，通常是在本地浏览器中存储。
- 5 5、当客户端在发送下一次HTTP请求时，它会将该cookie信息附加到请求头中，以便服务器可以通过该session ID来识别客户端。
- 6 6、服务器使用session ID来检索存储在服务器端存储器中的与该客户端相关的session数据，从而在客户端和服务端之间共享数据。
- 7
- 8 session\_start(): 启动会话，用于开始或恢复一个已经存在的会话。
- 9 \$\_SESSION: 用于存储和访问当前会话中的所有变量。
- 10 session\_destroy(): 销毁当前会话中的所有数据。
- 11 session\_unset(): 释放当前会话中的所有变量。
- 12 Session存储路径: PHP.INI中session.save\_path设置路径

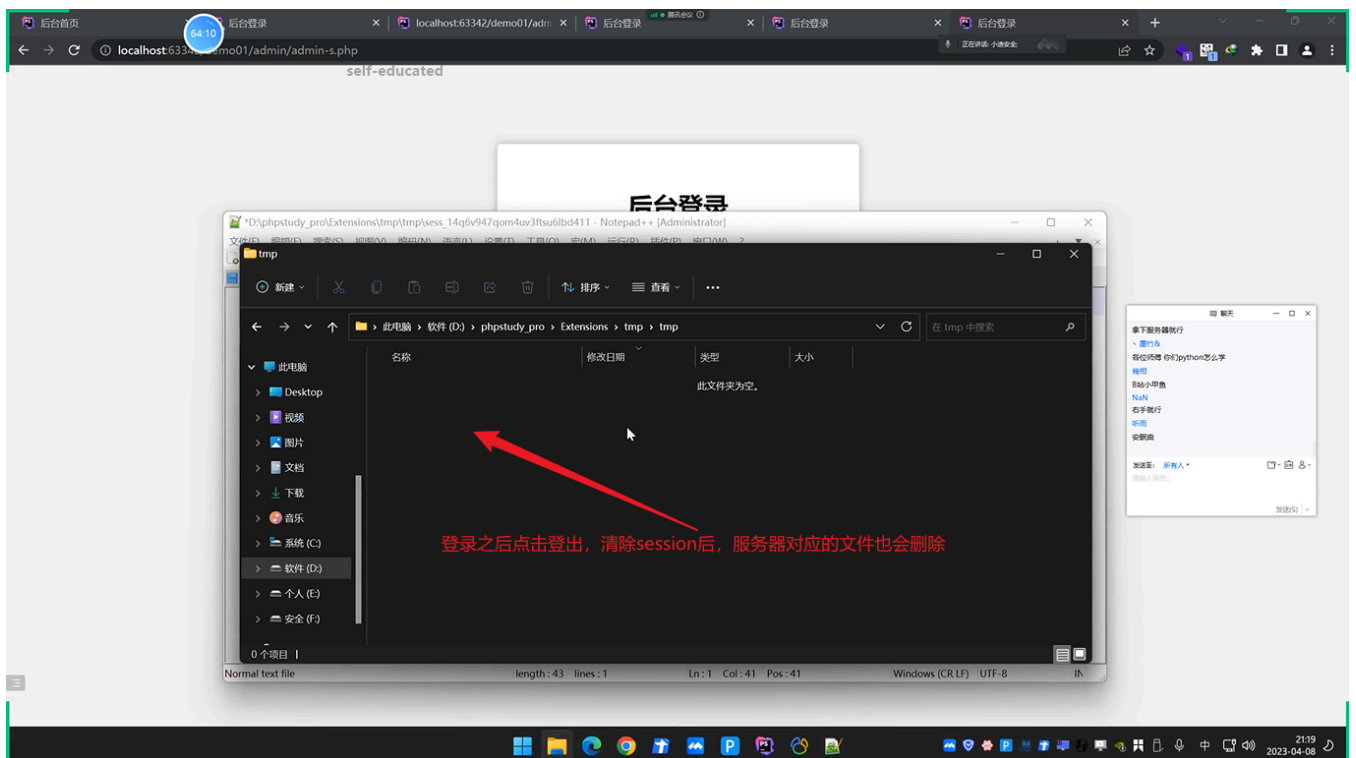
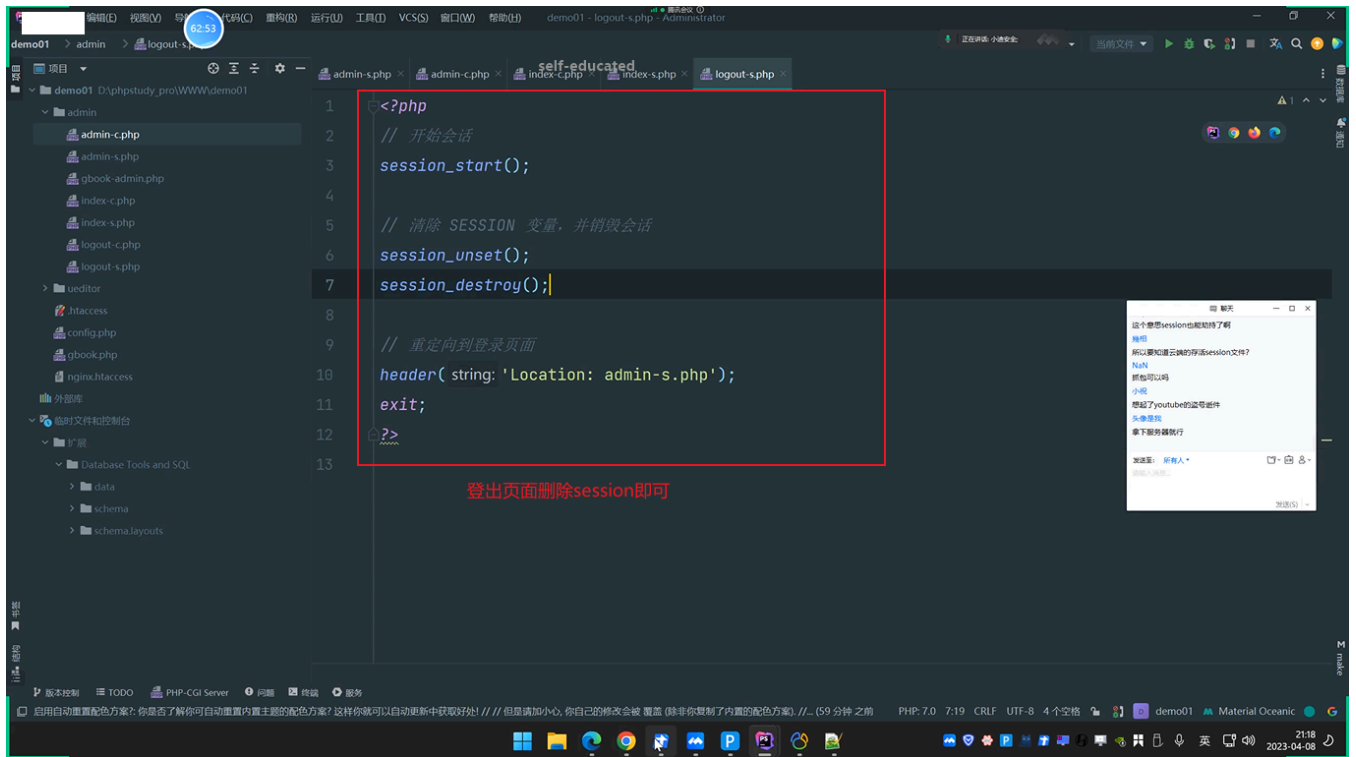
(1) 后台模块-身份验证:

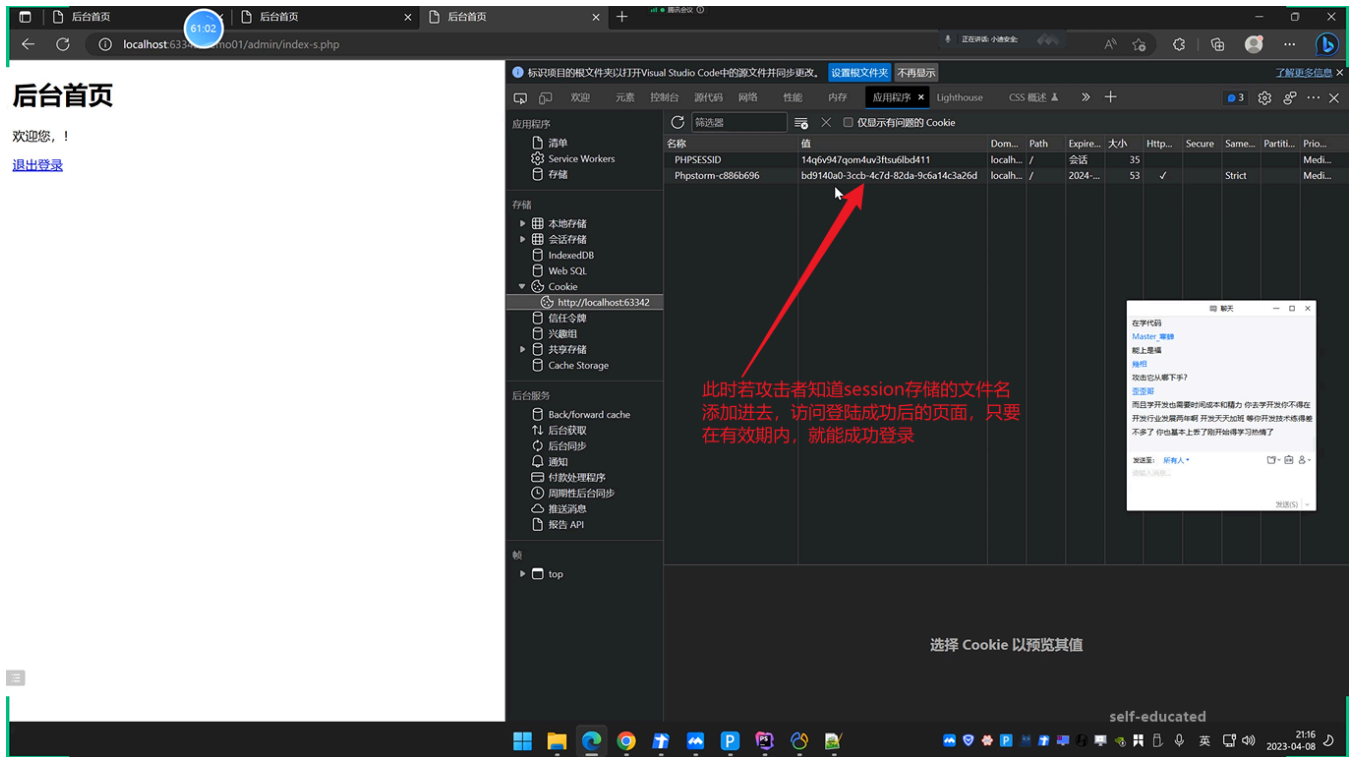








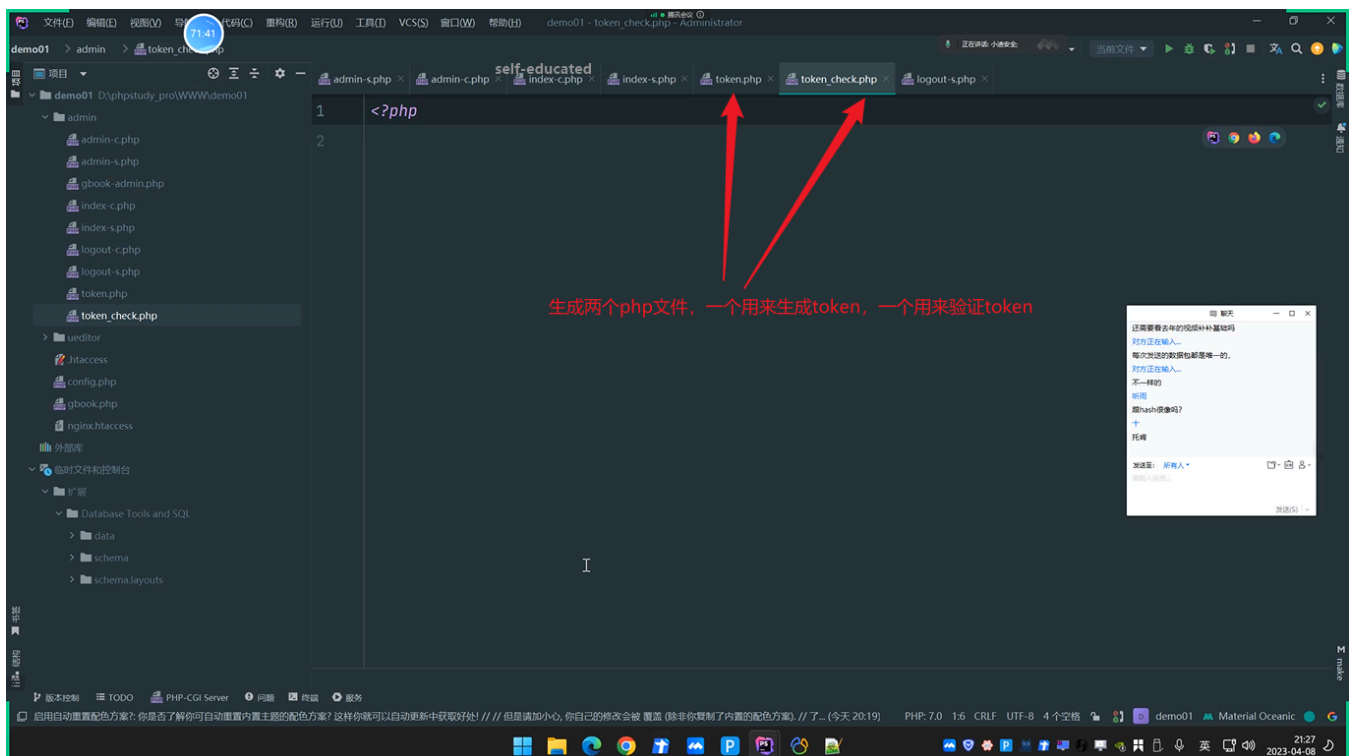


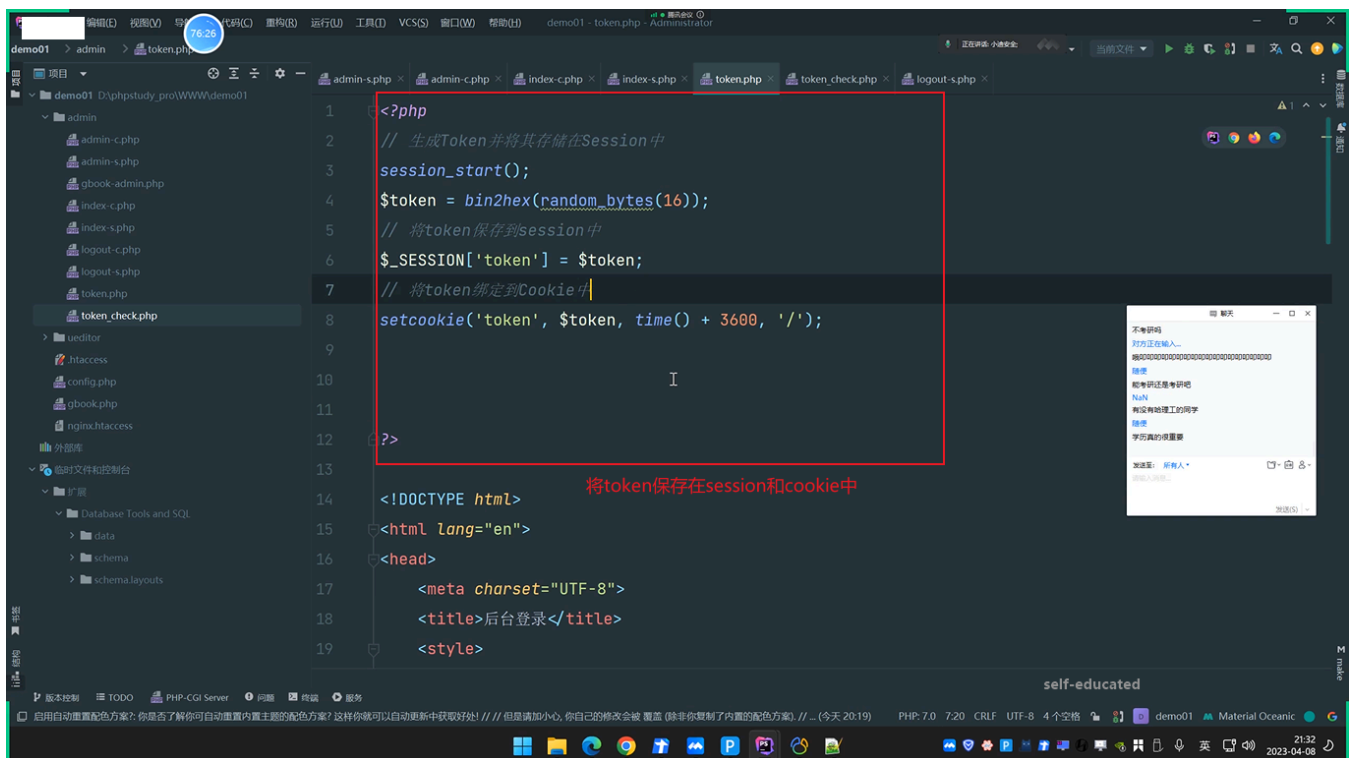
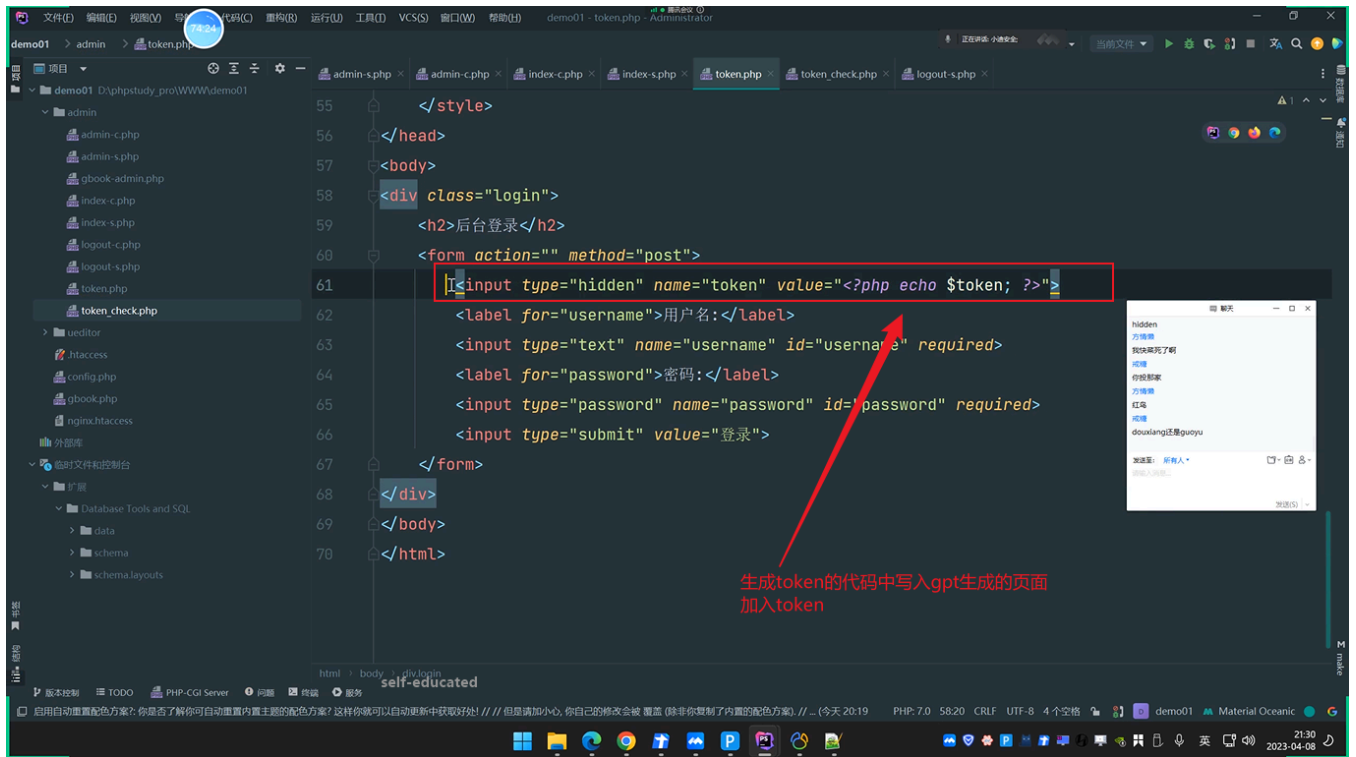


## 2.3 唯一性判断-Token使用

- 1、生成Token并将其存储在Session
- 2、生成Token并将其绑定在Cookie触发
- 3、尝试登录表单中带入Token验证逻辑
- 4、思考Token安全特性

### (1) Token使用：





demo01 - token\_check.php - Administrator

```
1 <?php
2 session_start();
3 $token = $_COOKIE['token'] ?? '';
4
5
6 if($_POST['username']='admin' && $_POST['password']='123456' && $_SESSION['token']==$token){
7     echo '登录成功!';
8 }else{
9     echo '登录失败!';
10 }
```

验证用户是否登陆成功

88.45

PHP: 7.0 9:18 CRLF UTF-8 4个空格 demo01 Material Oceanic 21:39 2023-04-08

demo01 - token\_check.php - Administrator

```
1 <?php
2 session_start();
3 $token = $_COOKIE['token'] ?? '';
4
5 if ($token == $_SESSION['token']) {
6     // token不匹配, 禁止访问
7     header( string: 'HTTP/1.1 403 Forbidden');
8     echo 'Access denied';
9     exit;
10 }else{
11     if($_POST['username']='admin' && $_POST['password']='123456'){
12         echo '登录成功!';
13         echo '你是管理员可以访问文件管理页面!';
14     }else{
15         echo '登录失败!';
16     }
17 }
```

还可以进行代码优化, 先做token验证, 再做账户密码验证

87.08

PHP: 7.0 11:16 (11 字符) CRLF UTF-8 4个空格 demo01 Material Oceanic 21:42 2023-04-08